Computer Science

Full Study Notes

Table of Contents

1. Computer Architecture 4
   1.1. Introduction 4
   1.2. CPU 4
      1.2.1. ALU 4
      1.2.2. CU 4
      1.2.3. CPU Characteristics 4
   1.3. Memory 4
      1.3.1. RAM 5
      1.3.2. HDD 5
   1.4. Converting data sizes (Decimal System - SI) 5
   1.5. Number Systems 6
      1.5.1. Base-10 6
      1.5.2. Base-2 6
      1.5.3. Relationship between base-2 and base-10 6
         1.5.3.1. Converting from Base-10 to Base-2 6
         1.5.3.2. Converting from Base-2 to Base-10 6
   1.6. Logic gates 7
      1.6.1. Introduction 7
         1.6.1.1. De Morgan's Law 7
      1.6.2. NOT 7
      1.6.3. AND 7
      1.6.4. NAND 7
      1.6.5. OR 8
      1.6.6. NOR 8
         1.6.6.1. De Morgan's Law 8
      1.6.7. XOR 8
2. Python 9
   2.1. Introduction 9
   2.2. Data Types 10
   2.3. Variables 10
   2.4. Operators 10
      2.4.1. Arithmetic operators 10
      2.4.2. Comparison operators 11
      2.4.3. Logical Operators On A Truth Table 11
      2.4.4. Operation + Assignment By Examples 11
   2.5. Loops 12
      2.5.1. A visual 12
      2.5.2. Example: Using lists to match two n-length lists 12
      2.5.3. Using lists without the range function 12
   2.6. Functions 13
      2.6.1. A visual 13
      2.6.2. Parameters & Arguments 13
      2.6.3. Returning Values 13
      2.6.4. Deterministic & Pure function 14
      2.6.5. Using functions to calculate the hypotenuse of a right triangle ABC 14

2.6.6. Functions to be aware of 14
2.6.6.1. print 14
2.6.6.2. int 14
2.6.6.3. range 14
2.7. Control Flow 14
2.7.1. A visual 15
2.7.2. If statements 15
2.7.3. Else statements 15
2.7.4. Using control flow to categorize age 16
2.8. Try it yourself 16
3. Complexity Theory 17
3.1. What is an algorithm? 17
3.1.1. Example: Sequential Search 17
3.1.2. Example: Binary Search 18
3.2. Time vs Space complexity? 19
3.2.1. Big-O complexity graph 19
3.2.2. Definition 19
3.2.2.1. Explanation 19
3.2.3. Ways of proving f(n) = O(g(n)) 20
3.2.4. Why is knowing the time complexity of an algorithm important? 21
4. Exam studying methodology 22

1. Computer Architecture

1.1. Introduction

Computer architecture is the structure of a computer system made from component parts.

1.2. CPU

- The CPU (**C**entral-**P**rocessing-**U**nit).
- It consists of many smaller units that handle different things, but in the explanation we will take, it will only contain an ALU 1.2.1 and a CU 1.2.2, both of their explanations will be simplified.
- It's essentially the brain of the computer, handling all the computation (we can call them tasks) that happen on the computer.



Figure 1: A physical CPU

,

1.2.1. ALU

- The ALU (**A**rithmetic-**L**ogic-**U**nit) is a component/unit of the CPU 1.2
- It handles all the arithmetic (addition, subtraction, division, multiplication, etc...)
- All the operations that happen inside the ALU are in base-2

1.2.2. CU

- The CU (**C**ontrol-**U**nit) is a component/unit of the CPU 1.2
- It will manage and execute instructions.
- For now, we will think of instructions as operations needed to perform certain actions on the computer (adding two numbers, opening an app, etc...)

1.2.3. CPU Characteristics

- A CPU has a clock speed: the speed at which cycles will execute, and it is a frequency.
- $1\text{GHz} = 1 * 10^9$ Hz

1.3. Memory

> Memory in the context of a computer system is the place where "data" will be stored. Data can be for example: images, text, files, Variables 2.3, etc...

We will discuss:

- Two types of memory: Primary Memory and Secondary Memory
- Data units: bit, byte, KB, MB, GB

Primary Memory	Secondary Memory
Fast access speed	Slower access speed
Volatile (data lost on restart)	Non-volatile (data persists)
Example: **RAM**	Example: **HDD, SSD**
Used for active tasks	Used for long-term storage

1.3.1. RAM

- RAM (**R**andom-**A**ccess-**M**emory) is a type of memory.
- RAM is primary memory.

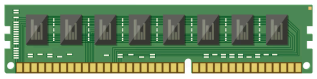


Figure 2: A physical stick of RAM

,

1.3.2. HDD

- HDD (Hard-Disk-Drive) is a type of memory.
- HDD is secondary-memory.



Figure 3: An HDD

1.4. Converting data sizes (Decimal System - SI)

Unit	Conversion
Bit	Smallest data unit
Byte	1 byte = 8 bits
Kilobyte (KB)	1 KB = 1,000 bytes
Megabyte (MB)	1 MB = 1,000 KB
Gigabyte (GB)	1 GB = 1,000 MB

1.5. Number Systems

1.5.1. Base-10

- Base-10 is the number system we humans use.
- It's made up of 10 digits, digits from (0 to 9): 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.
- Any other number is made up from these `10` digits.

1.5.2. Base-2

- Base-2 is the number system computers use.
- It's made up of two digits: `0` and `1`.
- Any other number is made up from these `2` digits.

1.5.3. Relationship between base-2 and base-10

Numbers can be converted to-and-back from base-2 to base-10.

1.5.3.1. Converting from Base-10 to Base-2

Example: Convert $(75)_{10}$ to binary

Step 1: Make a table.

Step 2: Divide 75 by 2 repeatedly.

Step 3: If the number is a decimal, take everything before the decimal and assign it the bit `1`; if the number is a whole number, write the number as is and assign it the bit `0`.

Step 4: Construct the number by reading the bits from bottom to top.

Step 5: If the number is now less than 1, record it, and stop.

number	index	bit
37	0	1
18	1	1
9	2	0
4	3	1
2	4	0
1	5	0
$\frac{1}{2}$	6	1

Table 1: Table for converting 75 to base-2.

1.5.3.2. Converting from Base-2 to Base-10

We will take the same example as above and in Table 1, and we will use the indexes to remake the base-10 number.

we had $(75)_{10} = (1001011)_2$

We need to only consider the "1"s and their indexes as shown in Figure 4

$(1001011)_2 = (2^0 + 2^1 + 2^3 + 2^6)_{10} = (75)_{10}$

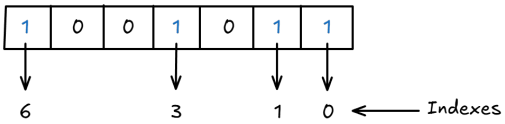


Figure 4: The 1's and their indexes.

1.6. Logic gates

1.6.1. Introduction

A logic gate is a device that performs a boolean function (logical operation). It is the smallest unit of computation in a CPU 1.2.

In this section, we will think of each logic gate as a device that has one or two inputs (NOT has one, all the others have two) and one output.

The primitive logic gates are: NOT, AND, OR.

From these we can build the derived gates: NAND, NOR, XOR. NAND and NOR are also called *universal* gates, because any other gate can be built using only NAND gates (or only NOR gates).

1.6.1.1. De Morgan's Law

$X = \neg(A \wedge B) \equiv \neg A \vee \neg B$

1.6.2. NOT

A NOT gate can be represented by the following table:

I_1	O_1
0	1
1	0

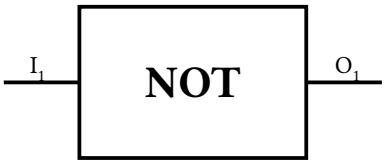


Figure 5: NOT

- The **NOT** gate outputs the complement of A, meaning 0 if A is 1, and 1 if A is 0.
- We will denote the operation: $\neg A = O_1$
- We can also denote the operation with a bar: $\overline{A} = O_1$

1.6.3. AND

An AND gate can be represented by the following table:

I_1	I_2	O_1
0	0	0
0	1	0
1	0	0
1	1	1



Figure 6: AND

- The **AND** gate outputs 0 for all pairs of (A, B), except when both **A** AND **B** are 1.
- We will denote the operation: $A \wedge B = O_1$

1.6.4. NAND

A NAND gate can be represented by the following table:

I_1	I_2	O_1
0	0	1
0	1	1
1	0	1
1	1	0



Figure 7: NAND

- The **NAND** gate outputs the opposite of what an AND gate would produce for a pair (A, B).

- We will denote the operation: $\neg(A \wedge B) = O_1$

1.6.5. OR

An OR gate can be represented by the following table:

I_1	I_2	O_1
0	0	0
0	1	1
1	0	1
1	1	1

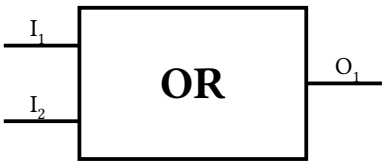


Figure 8: OR

- The **OR** gate outputs 1 if **A** OR **B** (or both) is 1, and 0 only when both are 0.
- We will denote the operation: $A \vee B = O_1$

1.6.6. NOR

A NOR gate can be represented by the following table:

I_1	I_2	O_1
0	0	1
0	1	0
1	0	0
1	1	0



Figure 9: NOR

- The **NOR** gate outputs the opposite of what an OR gate would produce for a pair (A, B), meaning it outputs 1 only when both **A** and **B** are 0.
- We will denote the operation: $\neg(A \vee B) = O_1$

1.6.6.1. De Morgan's Law

$X = \neg(A \vee B) \equiv \neg A \wedge \neg B$

1.6.7. XOR

An XOR (exclusive OR) gate can be represented by the following table:

I_1	I_2	O_1
0	0	0
0	1	1
1	0	1
1	1	0



Figure 10: XOR

- The **XOR** gate outputs 1 when **A** and **B** are different, and 0 when they are the same. Unlike OR, it outputs 0 when both inputs are 1.
- We will denote the operation: $A \oplus B = O_1$

2. Python

2.1. Introduction

Definition: Python is a “high-level”, “interpreted”, “general-purpose” scripting language.

Characteristic	Explanation
High-Level	Built on many lower-level abstractions. Python provides an easy-to-use language by building upon harder programming concepts and technologies.
Interpreted	An external program (the interpreter) reads and executes instructions line by line. Unlike compiled languages, Python code is interpreted at runtime.
General-Purpose	Can be used for most use-cases. Python is versatile: web development, data science, automation, etc.
Sequential	Executes instructions from top to bottom by default.
Imperative	Uses imperative programming style to maintain state.

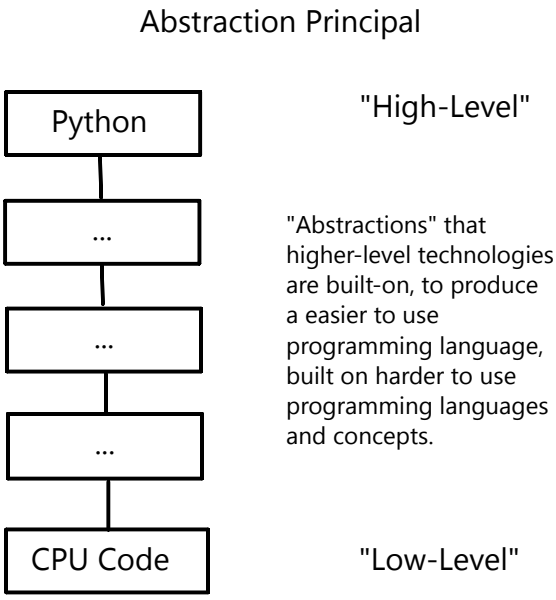


Figure 11: Compiled languages process.

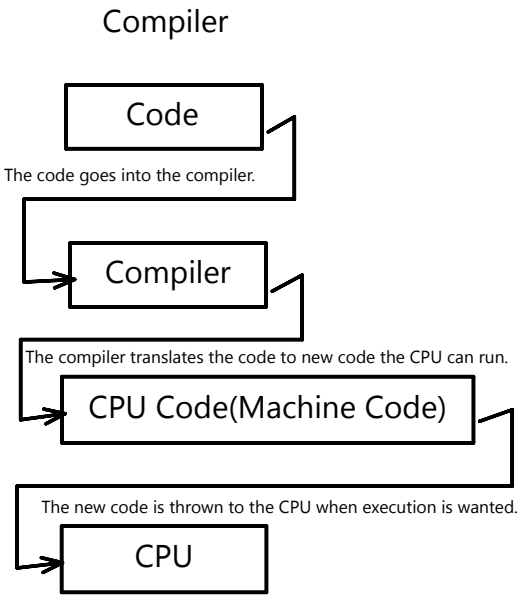


Figure 12: Abstractions.

2.2. Data Types

Type	Description & Examples
`int`	Integer - a whole number. Examples: `10`, `25`, `300`, `-5`
`float`	Decimal number. Examples: `52.6`, `19.3`, `1.0`, `5.0`
`bool`	Boolean - either `True` or `False`
`str`	String - sequence of characters. Examples: `"Hello"`, `"Text"`, `""`
`list`	Sequence of elements. Example: `[10, 5.2, 9.8]` Lists are heterogeneous and 0-indexed List indexing is non-commutative
`dict`	Dictionary - key-value store. Example: `{"name": "John", "age": 30}` Access: `person["name"]` ⟶ `"John"`
`None`	Represents a null value

2.3. Variables

Variables in Python are **mutable** (can change) and **dynamically typed** (no specific type restriction).

examples:

```
name = "Johnny"
johnny_age = 20
grades = [10, 5.2, 9.8, 12.5, 13.2]
person = {"name": "Johnny", "age": johnny_age}
```

2.4. Operators

2.4.1. Arithmetic operators

Operator	Name	Example
+	Addition	`10 + 5 = 15`
-	Subtraction	`10 - 5 = 5`
*	Multiplication	`10 * 5 = 50`
/	Division	`10 / 5 = 2.0`
**	Exponentiation	`3 ** 4 = 3 * 3 * 3 * 3 = 81`

Important: Division always returns a float: `int / int → float`

2.4.2. Comparison operators

Operator	Name	Result
==	Equality	`10 == 10 ⟶ True`
!=	Inequality	`10 != 5 ⟶ True`
>	Greater than	`10 > 5 ⟶ True`
<	Less than	`10 < 5 ⟶ False`
>=	Greater or equal	`10 >= 10 ⟶ True`
<=	Less or equal	`5 <= 10 ⟶ True`

2.4.3. Logical Operators On A Truth Table

p	q	**Conjunction** p **and** q $p \wedge q$	**Disjunction** p **or** q $p \vee q$	**Negation not** p $\neg p$	**Negation not** q $\neg q$
True	True	True	True	False	False
True	False	False	True	False	True
False	True	False	True	True	False
False	False	False	False	True	True

2.4.4. Operation + Assignment By Examples

All the arithmetic, and bitshift operators have an assignment equivalent used to modify variables.

```
# We will declare these variables for example purposes
first_number = 10
second_number = 5
first_string = "John"
second_string = "Johnny"

first_string += " Jason" # first_string will be "John Jason" after that, because "John"
+ " Jason" = "John Jason".
second_number -= 6 # second_number will be -1 after that, because 5 - 6 = -1.
second_number *= -2 # second_number will be 2 after that, because (-1)(-2) = 2.
first_number /= 5 # first_number will be 2.0 after that, because 10 / 5 = 2.0.
```

```
first_number = 5 # first_number will be 5 after that, normal assignment.
first_number <<= 2 # first_number will be 20 after that, because 5 << 2 = 20
first_number >>= 3 # first_number will be 2 after that, because 20 >> 3 is 2.
```

2.5. Loops

Loops are used for repeated actions.

2.5.1. A visual

```
accumulator = 0 # A variable that accumulates

for current_number in range(0, 10): # A loop over the range 0 to 10 (10 exclusive), and
current_number is the current value from the range
  accumulator += current_number # Mutating accumulator and adding current_number to it
```

After this, `accumulator` will be $0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 = 45$.

2.5.2. Example: Using lists to match two n-length lists

```
names  = ["John", "Johnnathan", "Jay", "Jane", "Jake"] # List of n names
grades = [9, 7.5, 6, 3.5, 6] # List of n grades
n      = 5

for i in range(0, n): # Looping n times
  name = names[i] # Accessing the i-th name from the names list
  grade = str(grades[i]) # Accessing the i-th grade from the grades list, and converting
it to a string
  print(name + " has a grade of " + grade) # Concatenating (Adding) the strings together
to make a sentence, here we see why we turned grade into a "str", because str + int and
str + float is not possible
```

The output is as shown below:

```
John has a grade of 9
Johnnathan has a grade of 7.5
Jay has a grade of 6
Jane has a grade of 3.5
Jake has a grade of 6
```

2.5.3. Using lists without the range function

Until now, we are still using the `range` function, but are **iterable** by default. An iterator allows any object to be **iterated over** by a loop. Let's compare:

```
nums = [1,2,3,4,5]
n = 5

for i in range(0, n):
  print(nums[i])
```

Code 1: Looping over lists with range

```
nums = [1,2,3,4,5]

for num in nums:
  print(num)
```

Code 2: Looping over lists with the list's iterative property

2.6. Functions

Functions should be thought of as machines that take an input and give us an output.



Figure 13: Function diagram

2.6.1. A visual

```
def add(x, y):
  return x + y
```

Code 3: A function that adds two numbers, x and y.

Now, to use this function we just **defined**, we will need to **call** it, as such:

```
result = add(10, 5)
print(result)
```

Code 4: Calling the function add

The output of result will now be `15`, since the function handled the addition of `x` and `y`, and `x` was `10`, and `y` was `5`.

2.6.2. Parameters & Arguments

- The parameters of a function is what we label the inputs. Consider Code 3, the parameters of that function are `x` and `y`.
- The arguments of a function is what is supplied for the labels of the inputs. Consider Code 4, the arguments are `10` and `5`.

2.6.3. Returning Values

The **return value** of a function is what the function outputs. Taking as example Code 3, the return value can be described as `x + y`, or also the sum of its inputs.

The return value is also what comes after the **return** keyword.

2.6.4. Deterministic & Pure function

Function Characteristic	Definition
Deterministic Function	A deterministic function is a function that returns the same outputs for the same inputs, and has no randomness in the determination of future states.

2.6.5. Using functions to calculate the hypotenuse of a right triangle ABC

```
def calculate_hypotenuse(AB, AC):
  BC = (AB**2 + AC ** 2) ** 0.5
  return BC
```

We can describe this function as follows:

- This function is called `calculate_hypotenuse`.
- It takes two parameters: AB and AC, which are supposed to be either ints or floats.
- It returns the hypotenuse of the triangle ABC.

2.6.6. Functions to be aware of

2.6.6.1. print

Output to the standard output (simplified).

```
>>> print("Hello World")
--> Hello World
```

2.6.6.2. int

Returns an integer constructed from a number or a string.

```
int(123.45) # 123
int("123") # 123
int("01110011", base=2) # 115
```

2.6.6.3. range

Range is technically a *class* not a function, but we will refer to it as a function. We will consider valid arguments to be integers.

Refer to Loops 2.5 to know how to use it.

2.7. Control Flow

Control flow describes how execution progresses from one command to the next.

2.7.1. A visual

```
your_age = 15
adult_age = 18

if your_age >= adult_age:
  print("You are an adult")
else:
  print("You are a minor")
```

Code 5: Control flow, if and else.

In the code above, we have two variables, `your_age` and `adult_age`.

We first compare `your_age` and `adult_age` using `>=` (Bigger than or equal to), which returns a bool type, this bool type is then considered by the **if** statement before it, and accordingly, the **branch** below if (the truthy-case branch) will be executed, otherwise, we are handling it with an **else** statement.

2.7.2. If statements

if executes it's branch if the statement given to it is a truthy value, a **truthy** value is a value that is not **falsy**.

Some **falsy** values in Python are: `None`, `0` and `False`

truthy values evaluate to **True** booleans, and **falsy** values evaluate to **False** booleans.

The structure of an **if statement** looks as such:

```
if boolean:
  branch instructions
```

2.7.3. Else statements

else statements are chained with **if** statements, and are executed in case the previous **if statement** turns out to be falsy, the structure is as such:

```
if boolean:
  branch instructions
else:
  branch instructions
```

One thing possible with **else** statements is chaining them with if statements, as such:

```
if boolean:
  branch instructions
elif boolean:
  branch instructions
elif boolean:
  branch instructions
else:
  branch instructions
```

Here, we have an if statement, chained with else statements, and a default case.

2.7.4. Using control flow to categorize age

```
age = 30

if age >= 60:
    print("You're a senior!")
elif age >= 40:
    print("You're an adult!")
elif age >= 20:
    print("You're a young adult!")
elif age >= 13:
    print("You're a teenager!")
elif age >= 3:
    print("You're a child!")
else:
    print("You're an infant!")
```

2.8. Try it yourself

You can go to the website: https://www.programiz.com/python-programming/online-compiler/, and follow everything each step of the way.

3. Complexity Theory

Complexity theory studies the relationship between an algorithm and the resource it takes.

3.1. What is an algorithm?

A computer algorithm is a sequence of steps a computer will take to achieve a particular outcome.

3.1.1. Example: Sequential Search

Given a sorted array A of length n, $(A_0 \leq A_1 \leq A_2 \leq ... \leq A_{n-1})$, and a number K, $K \in A$ sequential search is an algorithm that will go through each index in the array one by one, starting with 0, and stops when it finds the element K in the array and return it's position.

For example, if I have the array $A = [1, 5, 9, 12, 60, 110, 120, 121, 500, 505]$ and the number $K = 60$, sequential search will start checking in order:

- Is $A_0 = 60$? No, keep going.
- Is $A_1 = 60$? No, keep going.

...

- Is $A_4 = 60$? Yes, return 4, the desired position.
- This algorithm has a time complexity of $O(n)$, where $n = \text{length of the array}$, we will explain why later, but intuitively, we can think it is $O(n)$ because if the number K was at the end of the array(worst case scenario), it will take n steps to find K

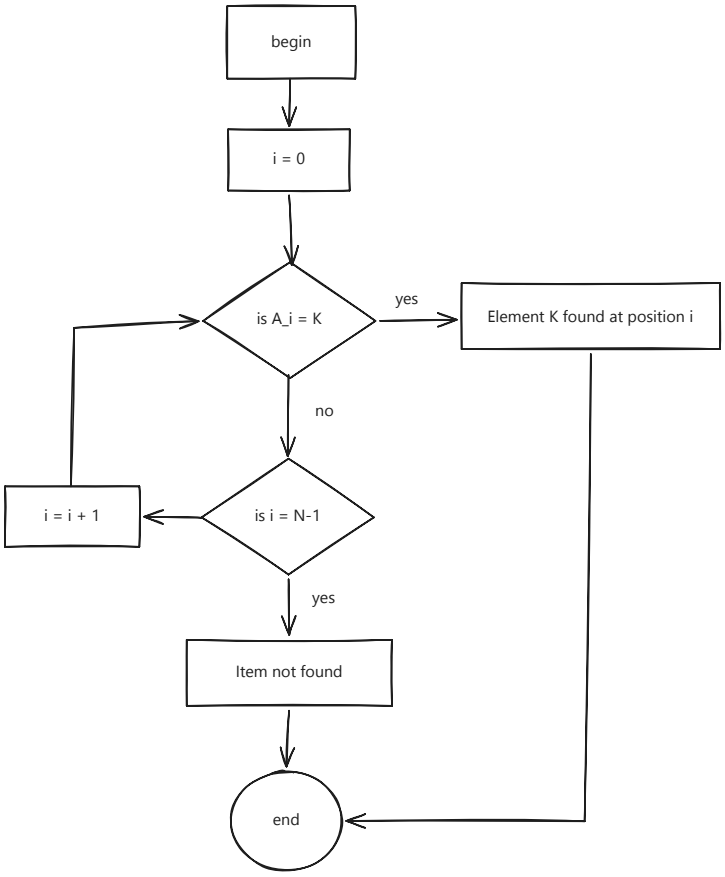


Figure 14: Sequential Search Diagram

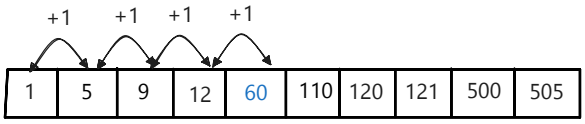


Figure 15: Sequential Search Array

3.1.2. Example: Binary Search

For binary search, we will consider the **sorted** 10-length array $A = [-5, -2, 0, 1, 2, 4, 5, 6, 7, 10]$ and $K = 7$, and we will start with low = 0, and high = 9, at each point, we will then repeatedly take midpoint $=$ low $+ \left\lfloor \frac{\text{high} - \text{low}}{2} \right\rfloor$, and call that point M, if $A_m > K$, then K is positioned before position m (the array is sorted), so we will take the left half, otherwise if $A_m < K$, then K is positioned after the position m, so we will take the right half, and repeat.

- This algorithm has $O(\log_2 n)$, but is written as simply $O(\log n)$ because 2 is a constant term we don't usually consider, this is because at each step in the algorithm, we cut the amount of work we have to do by half.

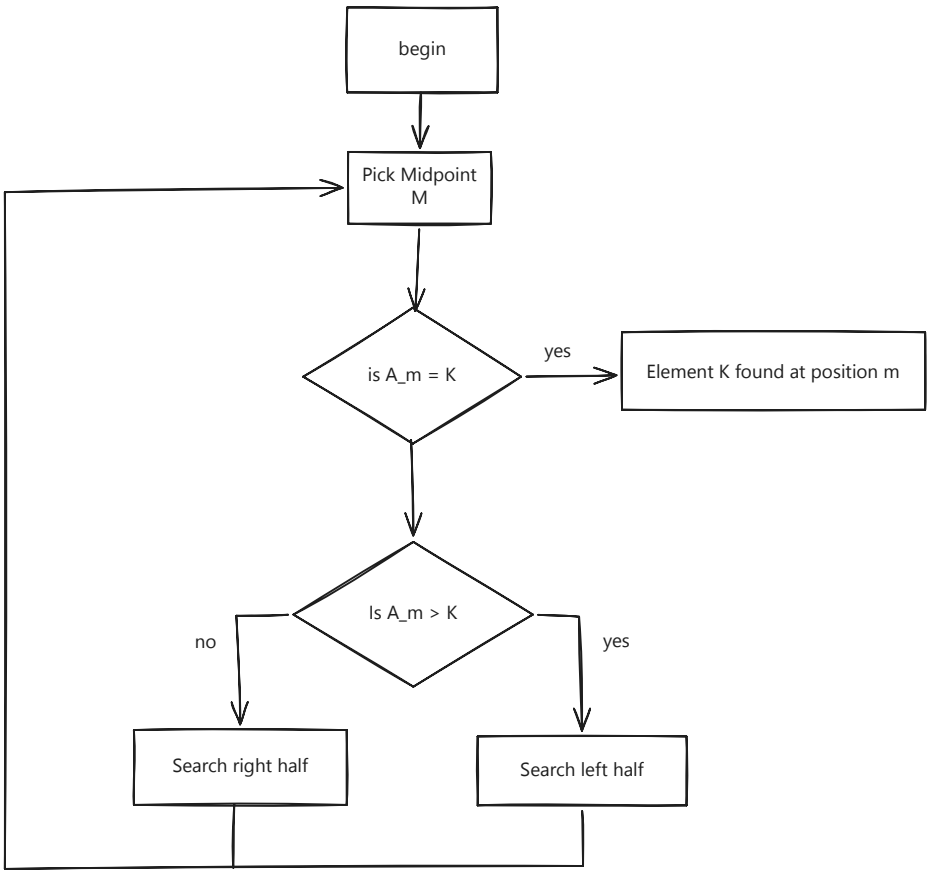


Figure 16: Binary Search Diagram

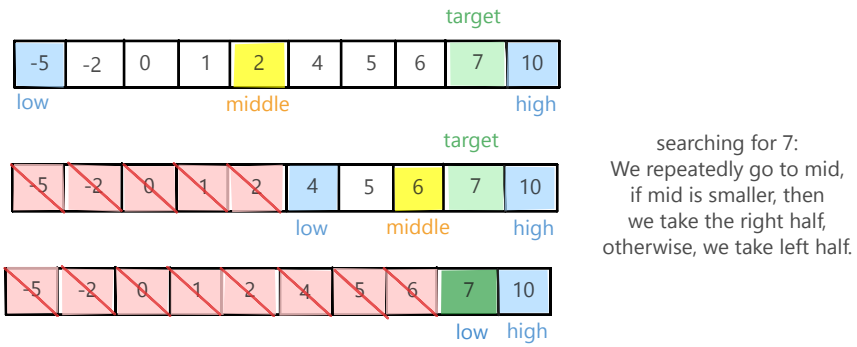


Figure 17: Binary Search Array

3.2. Time vs Space complexity?

- Time complexity describes how the execution time(number of steps) of an algorithm grows as the size of it's input grows.
- Space complexity describes how much more space an algorithm will take as the size of it's input grows.

In this course we will only be interested in time complexity.

3.2.1. Big-O complexity graph

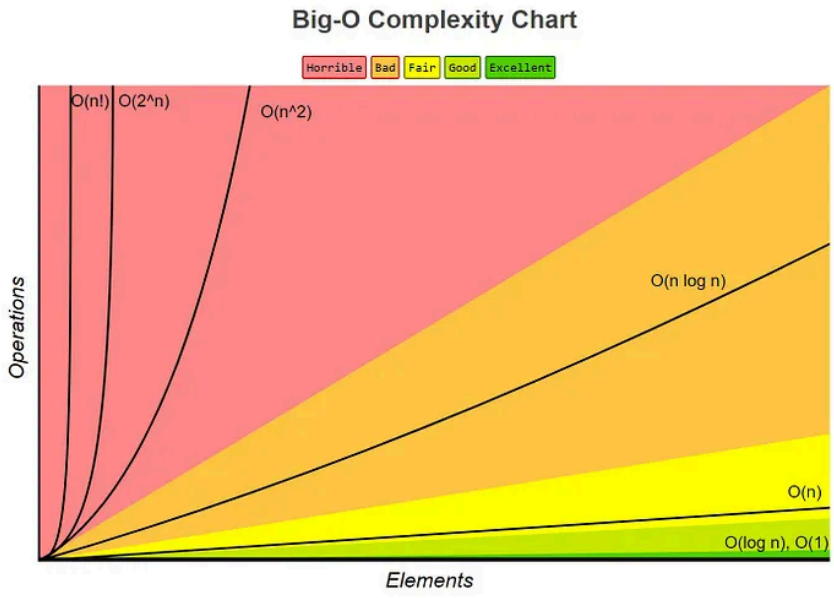


Figure 18: Big-O Complexity Chart

These are the standard time complexities we usually encounter, but we will only talk about $O(1)$, $O(n)$ and $O(n^2)$, it is important to note here that $n \geq 0$, and no negative input size exists.

3.2.2. Definition

$$f(n) = O(g(n)) \Leftrightarrow \exists c > 0, \exists n_0 > 0, \text{such that}$$
$$f(n) \leq c * g(n) \quad \forall n \geq n_0$$

$f(n)$ is $O(g(n))$ if there exists a $c > 0$ and an $n_0 > 0$, such that $f(n) \leq c * g(n)$ for all values of $n \geq n_0$

3.2.2.1. Explanation

Big-O describes the "upper-bound" of time an algorithm can take. Therefore, Big-O is always at most $c * g(n)$ for all $n \geq n_0$.

There exists other complexity-notations:

Notation	Symbol	Meaning
Big-O	O	Upper Bound
Little-o	o	Strict upper bound
Big-Omega	Ω	Lower bound
Little-omega	ω	Strict lower bound
Big-Theta	Θ	Upper + Lower bound

The following would be true for Big-O, but not for little-o:

$$x^2 \in O(x^2)$$

$$x^2 \in O(x^2 + x)$$

$$x^2 \in O(200x^2)$$

We will only be responsible for **Big-O** in this course.

A tip: You can look at the highest degree of the function, and already be aware of what time complexity it is .

3.2.3. Ways of proving f(n) = O(g(n))

Substitution	Rebuilding
• If we want to prove $f(n) = n^2 + 5n + 3 = O(n^2)$ • We identify $g(n) = n^2$ We will need, by definition: $n^2 + 5n + 3 \leq c * g(n)$ $n^2 + 5n + 3 \leq c * n^2$ • For all $n \geq 1$, we have: $n \leq n^2$ $5n \leq 5n^2$ $3 \leq 3n^2$ • And since these are all bigger than the terms in $f(n)$, we can say:	• If we want to prove $f(n) = n^2 + 5n + 3 = O(n^2)$ • We will take: $n_0 = 5$ $n \geq n_0$ $n \geq 5$ $5 \leq n$ • Multiplying by n, we get the middle term: $5n \leq n^2$ • For the constant term, since $n \geq 5$ we have $3 \leq n$, and $n \leq n^2$, so: $3 \leq n^2$

$n^2 + 5n + 3 \leq n^2 + 5n^2 + 3n^2$ $n^2 + 5n + 3 \leq 9n^2$ • So using $c = 9$ and $n_0 = 1$, we got a solution.	• And we replace: $n^2 + 5n + 3 \leq n^2 + n^2 + n^2$ $n^2 + 5n + 3 \leq 3n^2$ So using $c = 3$ and $n_0 = 5$, we got a solution.

3.2.4. Why is knowing the time complexity of an algorithm important?

Let's imagine a sorted array A like we did before, and let's say that array has a length of $N = 10^9$ (1 billion elements).

Using **sequential search** $O(n)$, would mean an upper-bound of operations $= o = 10^9$ operations (billion operations), to estimate how long that takes, we count CPU cycles per micro-operation:

Operation	Theoretical CPU Cycles
Load $A[i]$ from memory	1
Compare $A[i] == K$	1
Branch on result (if true stop, if false continue)	1
Increment i	1
Check if $i < n$ and branch	1

These are $c \approx 5$ cycles, and the time for the n operations in relation with clock speed and 5-cycles:

$$T(o) = \frac{\text{cycle count} * \text{operation count}}{\text{Clock Speed Frequency}} = \frac{5o}{f}$$

Where f is the clock-speed of the CPU (the speed of operation cycling) in "Hz". and o is the operation count. If we consider a theoretical CPU of clock speed 0.295 GHz (Same speed as a PlayStation 2).

$$T(o) = T(10^9) = \frac{5(10^9)}{(0.295)(10^9)} \approx 16.94 \text{ s}$$

It would take 16.94 seconds on a 0.295GHz CPU to execute the algorithm.

On the other hand, if we considered **binary search** instead with: $N = 10^9$, we would have an upper bound of:

$$o' = \log_2 N = \log_2 10^9 \approx 30$$

And so:

$$T(o') = T(\log_2 N) = \frac{5(\log_2 10^9)}{(0.295)(10^9)} \approx 5 * 10^{-7} \text{ s} \approx 0.0000005 \text{ s}$$

Comparing the two times: $\frac{T(o)}{T(o')} = \frac{16.94}{5*10^{-7}} \approx 33880000$, that's 33 million times faster. Now, realistically, there is one more operation cycle binary search does (calculating the midpoint), but I have omitted that in this calculation. This improvement in runtime cost is an **asymptotic improvement**

If I were to consider the actual CPU cycles per operations of binary search, the result would be like:

$$T_{\text{seq}}(o) = \frac{5o}{f} \qquad T_{\text{bin}}(o) = \frac{6o}{f}$$

$T_{\text{seq}}(o) = T_{\text{seq}}(10^9) = \frac{5(10^9)}{(0.295)(10^9)} \approx 16.94 \text{ s}$ $T_{\text{bin}}(o') = T_{\text{bin}}(\log_2 N) = \frac{6(\log_2 10^9)}{(0.295)(10^9)} \approx 0.0000006 \text{ s}$

(T_{bin} has 6 cycles per operation, one extra for calculating the mid-point)

Which still, isn't a big difference from the original estimation.

4. Exam studying methodology

- Focus on understanding the concepts before memorizing them, there's not even the need to memorize them.
- 20% of the effort yields 80% of the outcome, don't try and nail all the details, just focus on the most important things.
- Focus on the English parts of things and try to explain things to yourself and other people to see if you understand.
- Regarding Python, you can try everything each step of the way, read Try it yourself 2.8

Good Luck! I want good grades.

Author: Michael Salloum